

MEMORIA DESCRIPTIVA

RotaStock Plus

*Control de inventario con valoración FIFO (PEPS),
clasificación ABC heurística, plan de compras,
conteos de almacén e informes ejecutivos*

Versión 2.0.0 · Escritorio Windows · 100 % offline

Autor

Pedro Giovanni Ricra Figueroa

2026

ÍNDICE

1. Introducción.....	3
2. Infraestructura tecnológica y arquitectura.....	4
2.1 Stack tecnológico	4
2.2 Estructura del proyecto.....	5
2.3 Persistencia, rutas de datos y copias.....	5
2.4 Aislamiento de procesos e IPC	6
2.5 Cascada de carga y modo navegador	6
3. Objetivos del sistema	7
4. Requisitos de operación.....	7
5. Arquitectura de la interfaz.....	8
6. Especificación de los nueve módulos.....	9
6.1 Módulo 01 — Panel de control.....	9
6.2 Módulo 02 — Productos.....	10
6.3 Módulo 03 — Clasificación ABC.....	11
6.4 Módulo 04 — Movimientos	12
6.5 Módulo 05 — Qué comprar.....	13
6.6 Módulo 06 — Conteos de almacén.....	14
6.7 Módulo 07 — Configuración.....	15
6.8 Módulo 08 — Copias de seguridad.....	15
6.9 Módulo 09 — Informes (PDF y Excel).....	16
7. Modelado matemático y algoritmos de negocio.....	17
7.1 Replay FIFO (PEPS)	17
7.2 Ejemplo numérico de capas	18
7.3 Rotación, cobertura y días de inventario	19
7.4 Semaforización por inactividad	19
7.5 Asignador heurístico ABC y estabilidad XYZ	20
7.6 Plan de compras (punto de pedido).....	21
7.7 Ajustes de conteo e IRA	22
7.8 Orden de recálculo	22
8. Flujo de operación	23
9. Directrices de operación y mantenimiento	24
10. Conclusiones.....	25
Anexo A. Esquema JSON de estado único.....	26

Anexo B. Correspondencia interfaz–motor	27
Anexo C. Validaciones de formularios.....	28
Anexo D. Estructura de los informes PDF y Excel.....	29
Anexo E. Canales IPC y empaquetado	30

1. Introducción

RotaStock Plus es una aplicación de escritorio para Windows, 100 % offline, pensada para operar un almacén completo: registrar productos, compras y ventas, valorar el inventario con FIFO (PEPS), clasificar SKU por importancia y por estabilidad de demanda, sugerir qué comprar, auditar existencias con conteos físicos y emitir informes ejecutivos en PDF y Excel.

El sistema no guarda indicadores precalculados como fuente de verdad. La única autoridad es la bitácora de movimientos. Cada alta, edición, baja o conteo dispara un replay cronológico: se reconstruyen lotes, stock, COGS, rotación, clasificación ABC, sugerencias de compra y alertas. Esa decisión elimina la deriva entre «lo que muestra la pantalla» y «lo que ocurrió en el almacén».

RotaStock Plus es la evolución del motor FIFO original: conserva el replay verificado y lo extiende a nueve módulos que cubren el ciclo del almacén, de la captura del movimiento al informe para la gerencia. Todo el estado vive en un único JSON (`inventario_completo`), portable 1:1 entre computadoras. La presente memoria describe la infraestructura, cada módulo, el modelado matemático, el flujo de operación y las reglas de mantenimiento.

El producto se entrega como instalador NSIS (RotaStockPlus Setup 2.0.0.exe) y como ejecutable portable (RotaStockPlus.exe). El frontend es una SPA HTML5/CSS/JavaScript Vanilla que también puede abrirse en el navegador, usando `localStorage` cuando no hay proceso Electron. Licencia MIT. Autor: Pedro Giovanni Ricra Figueroa.

2. Infraestructura tecnológica y arquitectura

La arquitectura es de tres capas locales, sin servidor: (1) el proceso principal de Electron administra ventana, archivos y copias; (2) el puente preload expone cuatro operaciones; (3) el renderer —HTML, CSS y un único `app.js`— contiene la interfaz y el motor de negocio (Engine). No hay frameworks, ni bundlers, ni CDN.

2.1 Stack tecnológico

Capa	Tecnología	Función
Escritorio	Electron 30 + electron-builder	Ventana nativa, NSIS e .exe portable
IPC	preload.js + contextBridge	load / save / listBackups / readBackup
Presentación	HTML5 + CSS3 (claro / oscuro)	SPA de nueve vistas

Motor	JavaScript Vanilla (Engine)	FIFO, ABC, pronóstico, métricas
Estado	Store (JSON)	Carga, guardado y metadatos
Gráficos	Chart.js 4 (vendedorizado)	Barras, dona y Pareto
PDF	jsPDF + autotable (vendedorizados)	Informe de 6 secciones
Excel	SheetJS (xlsx)	Libro de 4 hojas con fórmulas vivas
Persistencia	JSON único + backups/	Fuente de verdad local, offline

Tabla 1. Stack tecnológico de RotaStock Plus.

Chart.js, SheetJS, jsPDF y autotable viajan dentro de frontend/assets/vendor/. El renderer no tiene nodeIntegration: solo un puente IPC con contextIsolation activo. Los números se formatean con locale en-US; las fechas, con locale es-PE. Los identificadores se generan con el prefijo (p, m, aud) más Date.now() en base 36.

2.2 Estructura del proyecto

- main.js — proceso principal: rutas de datos en AppData, IPC, backups diarios, ventana 1 360 × 860 (mínimo 1 024 × 640), barra de menú oculta, icono build/icon.ico.
- preload.js — expone window.rotastock = { isElectron, loadDB, saveDB, listBackups, readBackup }.
- package.json — productName RotaStockPlus, targets nsis + portable x64, atajos de escritorio e inicio, oneClick = false.
- frontend/index.html, style.css, app.js — interfaz, tema y motor de los nueve módulos.
- frontend/assets/vendor/ — chart.umd, xlsx.full, jspdf.umd, jspdf.plugin.autotable.
- datos/inventario.json — semilla de desarrollo; en producción se copia una sola vez al perfil del usuario.
- backups/ — copias automáticas diarias (ignoradas por git).
- build/icon.ico e icon.png — icono del ejecutable, del instalador y de la SPA.
- LICENSE — MIT, copyright 2026 Pedro Giovanni Ricra Figueroa.

2.3 Persistencia, rutas de datos y copias

Instalado, los datos viven en C:\Users\<usuario>\AppData\Roaming\RotaStockPlus\ (datos\inventario.json y backups\). En desarrollo se usa la misma ruta de AppData, no la carpeta del proyecto: getBasePath() apunta siempre a app.getPath('appData') + RotaStockPlus, de modo que el .exe portable y el instalador no escriben dentro del app.asar (solo lectura).

El esquema de estado es un único objeto { meta, config, productos, movimientos, conteosCiclicos }. Si el archivo no existe, se copia la semilla del proyecto o se crea defaultDB(). Un fallo de lectura no bloquea el arranque: se escribe una base vacía y se continúa. Cada guardado actualiza inventario.json y, la primera vez del día, escribe

inventario-YYYY-MM-DD.json. Las copias con más de 30 días (por mtime) se eliminan solas. La migración entre PCs se hace con «Descargar copia completa» / «Cargar copia» (JSON), no con el PDF ni el Excel.

Los lotes FIFO no se persisten. Viven solo en memoria durante Engine.compute(). Los campos `_cogs` y `_ingreso` de cada movimiento se reescriben en cada replay y no son fuente de verdad. Los campos `claseABC` y `categoriaRotacion` que puedan aparecer en una semilla de desarrollo son decorativos: la clase y el estado se calculan siempre.

2.4 Aislamiento de procesos e IPC

El renderer corre con `contextIsolation: true` y `nodeIntegration: false`. No puede leer el disco ni `require('fs')`. Las cuatro operaciones pasan por `ipcMain.handle`:

Canal	Dirección	Contrato
db:load	renderer → main	Devuelve el texto UTF-8 de inventario.json; si falla, escribe defaultDB y lo devuelve.
db:save	renderer → main	Escribe el texto completo; dispara maybeBackup; retorna true/false.
db:listBackups	renderer → main	Lista *.json de backups/, orden inverso (más reciente primero).
db:readBackup	renderer → main	Lee un archivo cuyo nombre se sanitiza con path.basename, para no salir de backups/.

Tabla 2. Canales IPC. El renderer nunca recibe un objeto fs, solo cadenas JSON.

2.5 Cascada de carga y modo navegador

Store.load() intenta, en este orden: (1) window.rotastock.loadDB() si isElectron; (2) localStorage['rotastockplus_db']; (3) fetch de ../datos/inventario.json o datos/inventario.json; (4) un objeto vacío con umbrales 7/15/30 y nombre «Mi negocio». Store.save() siempre escribe localStorage y, si hay Electron, también el disco. Así el mismo frontend funciona empaquetado y abierto como archivo HTML.

El tema se aplica antes del primer pintado: un script en el <head> lee rotastock_theme (light | dark) y fija data-theme, para no parpadear. applyTheme() sincroniza ese valor con config.preferenciasUI.tema (claro | oscuro). En el navegador no hay copias automáticas de disco: el módulo 08 lo indica. El PDF, el Excel y el JSON de copia completa sí funcionan en ambos modos.

3. Objetivos del sistema

1. Valorar cada venta con FIFO estricto, reconstruyendo lotes desde la bitácora.
2. Diagnosticar rotación, cobertura, días de inventario y capital dormido en un período configurable.
3. Clasificar productos A/B/C con un asignador heurístico por proximidad y medir estabilidad XYZ.
4. Sugerir cantidades de compra con pronóstico, reserva de seguridad y punto de pedido.
5. Auditar el almacén con conteos físicos, ajustes automáticos, edición y reversión.
6. Emitir un PDF ejecutivo de seis secciones y un Excel de cuatro hojas alineados al mismo período.
7. Respalidar y restaurar el estado completo en un solo archivo JSON, sin servidor ni red.
8. Operar 100 % offline, con tema claro/oscuro y el mismo motor en escritorio o navegador.

4. Requisitos de operación

Componente	Requisito mínimo
Procesador	Intel Core i3 o equivalente
Memoria RAM	4 GB (8 GB recomendados al empaquetar)
Pantalla	1 024 × 640 px; diseño pensado para 1 360 × 860
Sistema operativo	Windows 10 o superior (instalador o portable)
Desarrollo	Node.js LTS (npm install, npm start, npm run build)
Conectividad	Ninguna en operación
Almacenamiento	Perfil de usuario escribible (AppData\Roaming\RotaStockPlus)

Tabla 3. Requisitos mínimos.

Empaquetado: npm run build produce el instalador NSIS (directorio cambiabile, accesos directos de escritorio e inicio) y RotaStockPlus.exe portable. El usuario final no necesita Node.js.

5. Arquitectura de la interfaz

SPA de dos columnas sobre paleta grafito cálido / papel, con acentos azul petróleo (#1f5668) y arcilla (#a85a32), y conmutador claro/oscuro persistido. La cabecera lleva marca RotaStock Plus, tema y botón Actualizar. El período de análisis vive en el panel de control (no en la cabecera global): 1 semana, 15 días, 1 mes (defecto), 3 meses, 6 meses, 1 año o Siempre. El

sidebar lista las nueve vistas y, al pie, el nombre del negocio, el sello FIFO (PEPS) y el conteo de productos y movimientos.

Un modal genérico cubre altas, ediciones y confirmaciones (Escape o clic en el overlay lo cierra). Un toast de 2,8 s confirma las operaciones. En anchos ≤ 900 px el menú se colapsa; el botón hamburguesa abre el sidebar. El tema también se cambia desde Configuración (Apariencia). Los nombres en tablas y tarjetas se escapan (esc) para no inyectar HTML. Chart.js hereda --muted y --grid del tema activo.

Hay tres períodos independientes: el del Panel (PERIOD, defecto 30 días), el de la bitácora de Movimientos (defecto Siempre) y el de Informes (defecto último mes). Cambiar uno no altera los otros. Clasificación ABC y «Qué comprar» se calculan siempre sobre el historial completo (compute(0)), para que un lente corto no mueva un SKU de clase ni dispare una compra ficticia.

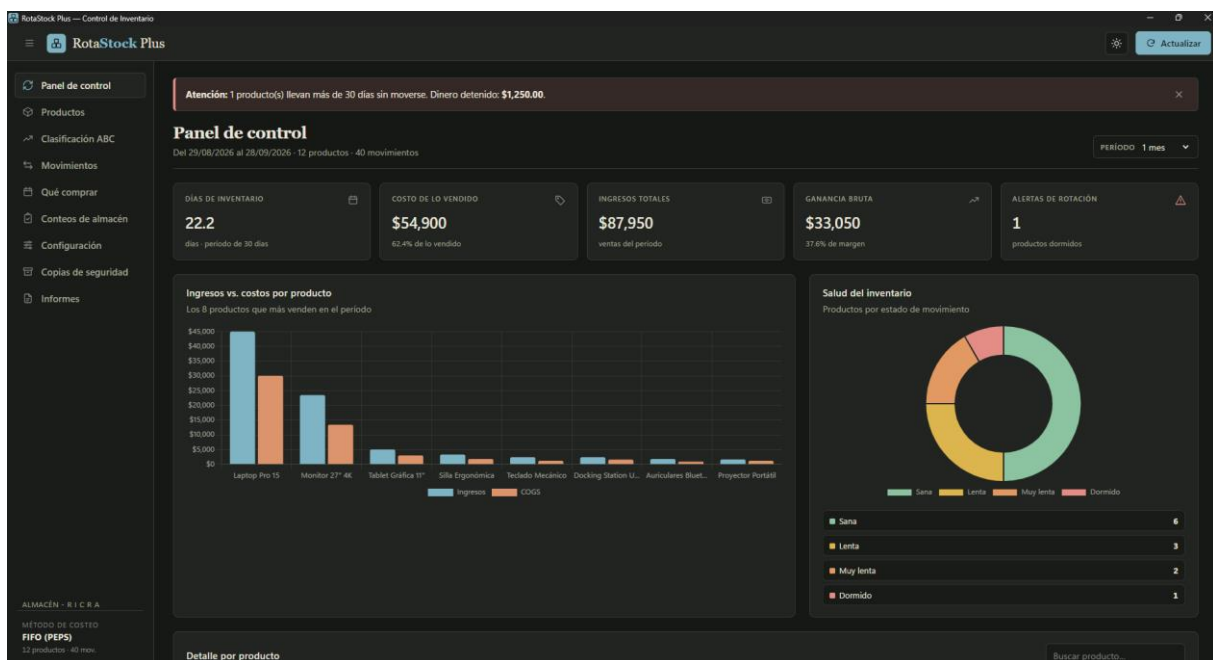


Figura 1. Panel de control: KPIs, gráficos y detalle por producto.

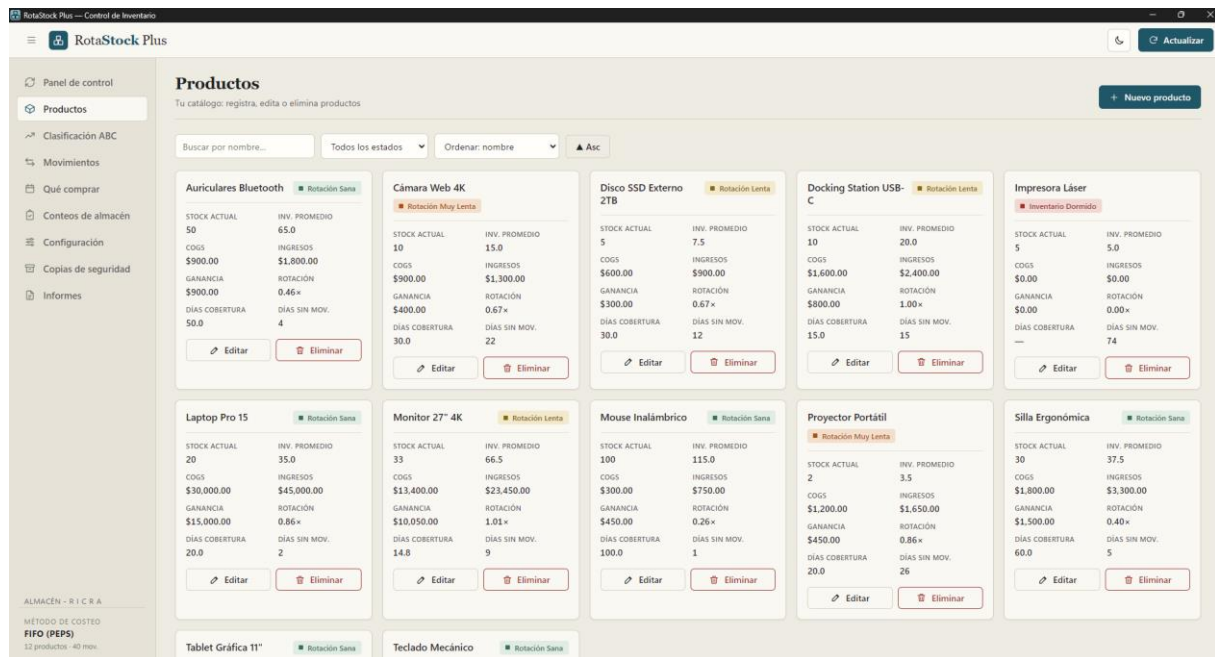


Figura 2. Catálogo de productos con métricas de rotación y acciones CRUD.

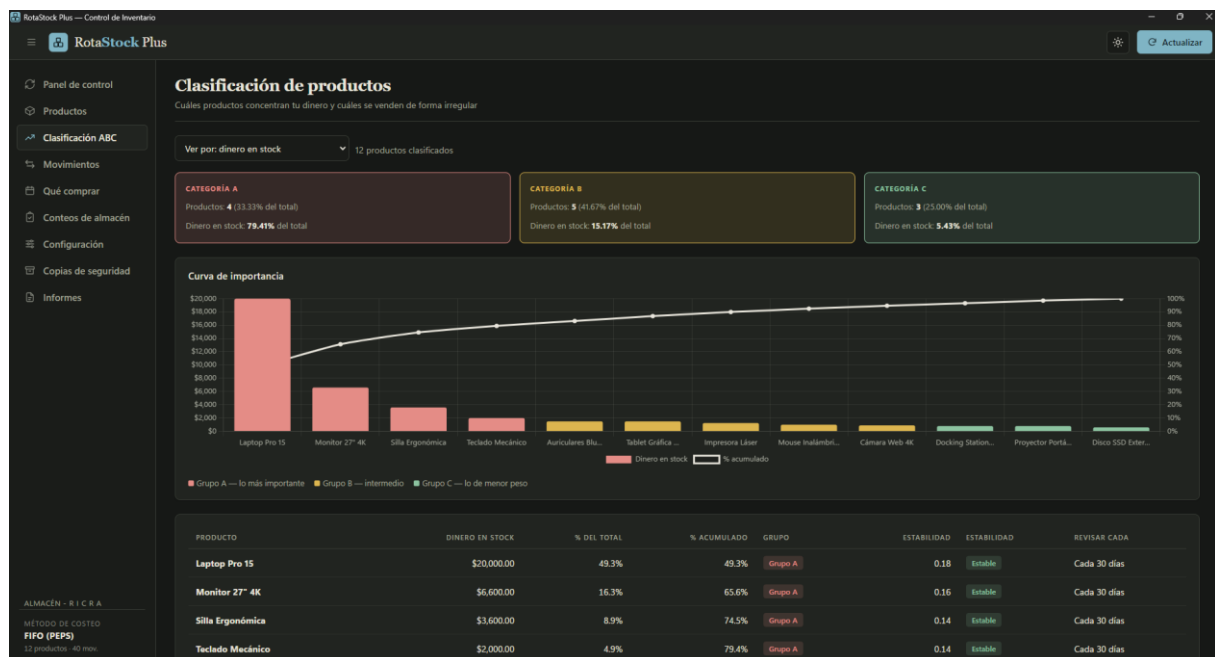


Figura 3. Clasificación ABC con curva de importancia y estabilidad XYZ.

6. Especificación de los nueve módulos

6.1 Módulo 01 — Panel de control

Vista de arranque. Un banner avisa cuántos productos superan el umbral de «muy lenta» y cuánto dinero está detenido; se cierra con × sin borrar el KPI. Cinco tarjetas:

- Días de inventario — DIO del período, con subtítulo «días · período de Δ días» o «sin ventas».
- Costo de lo vendido — COGS y su peso porcentual sobre los ingresos.
- Ingresos totales — ventas del período (cantidad $\times$ precio).
- Ganancia bruta — ingresos – COGS, con margen %.
- Alertas de rotación — conteo de SKU en estado Dormido.

El gráfico de barras compara ingresos y COGS de los ocho productos de mayor ingreso (nombres recortados a 18 caracteres). La dona (cutout 62 %) y su leyenda cuentan SKU por estado (Sana, Lenta, Muy lenta, Dormido). La tabla «Detalle por producto» muestra inventario promedio en unidades, rotación, cobertura, días sin movimiento y píldora de estado, con búsqueda, orden por columna (defecto: días sin movimiento descendente) y paginación de 8 filas. El pie del período se escribe en locale es-PE.

6.2 Módulo 02 — Productos

Rejilla de tarjetas con stock actual, inventario promedio, COGS, ingresos, ganancia, rotación, cobertura y días sin movimiento, más Editar y Eliminar. Barra: búsqueda, filtro por estado y orden por siete campos (nombre, rotación, COGS, ingresos, ganancia, cobertura, días sin movimiento), con dirección ascendente/descendente.

Alta

Nombre obligatorio; costo base; unidades iniciales; costo total de esas unidades; tiempo de entrega del proveedor (días, defecto 7). Si hay stock inicial, el costo total es obligatorio y se inserta una compra con unitario = total / unidades (cuatro decimales). El producto nace con `desviacionLeadTime = 1` y `nivelServicioZ = 1,65` (Alto).

Edición y baja

Edición: nombre, lead time, margen de seguridad Z (Alto 1,65 / Normal 1,28 / Bajo 1,04) y costo base. Baja: confirma cuántos movimientos se perderán, elimina el SKU y todos sus movimientos, y recalcula. El maestro no guarda un campo stock: el stock es siempre derivado del replay. El costo base solo se usa cuando no hay capas vivas (faltante de conteo sobre stock 0).

6.3 Módulo 03 — Clasificación ABC

El operador elige la base: dinero en stock (valor) o costo de lo vendido (COGS). Tres tarjetas resumen A, B y C (conteo, % de SKU y % de la base). Una curva de importancia combina barras por producto (coloreadas por grupo: A rojo, B ámbar, C verde) con la línea de %

acumulado en eje derecho 0–100. La tabla lista valor o COGS, % del total, % acumulado, grupo, coeficiente de variación, etiqueta de estabilidad (Estable / Variable / Irregular) y la cadencia de revisión sugerida: A cada 30 días, B cada 60, C cada 120.

Si la base es COGS y un SKU no vendió, se le asigna un $\varepsilon = 0,001 \times \text{valor en stock}$ para no dejarlo en cero absoluto. La clasificación se calcula sobre el historial completo (compute(0)), no sobre el período del panel.

6.4 Módulo 04 — Movimientos

Bitácora de compras y ventas. Filtros: búsqueda por nombre, tipo (todas / compras / ventas; los ajustes de sobrante se agrupan con compras) y período propio (Hoy, semana, 15 días, mes, 3 meses, medio año, año, Siempre). Columnas: fecha y hora, producto, tipo (con motivo de ajuste si existe), cantidad entera, unitario, total y acciones. Paginación de 10, máximo 9 botones visibles. Orden por encabezado (defecto: fecha descendente).

El formulario exige producto elegido de autocompletado (hasta 8 coincidencias), tipo, fecha/hora local (datetime-local), cantidad ≥ 1 y costo o precio ≥ 0 . El rótulo cambia: «Costo de compra» o «Precio de venta». En ventas se valida el stock FIFO; al editar se reintegra la cantidad previa (si era venta se suma, si era compra se resta). Una venta que exceda el disponible se rechaza con el disponible redondeado. Toda persistencia dispara el replay: se reescriben m._cogs y m._ingreso. Eliminar un movimiento no pide confirmación modal: lo quita y recalcula.

6.5 Módulo 05 — Qué comprar

Tabla de sugerencias ordenada por alerta (primero los que hay que comprar) y, en empate, por holgura respecto al punto de pedido. Columnas: producto (con lead time en texto «El proveedor tarda $\sim N$ días»), stock, venta promedio/día, pronóstico mensual (suavizado exponencial $\alpha = 0,3$), reserva de seguridad, «pedir al llegar a» (punto de pedido), cantidad a pedir y estado (Comprar / Bien). Un banner resume cuántos SKU hay que comprar y el total de unidades sugeridas, o confirma que ninguno está bajo el punto de pedido.

6.6 Módulo 06 — Conteos de almacén

Tres KPIs: exactitud de conteos (IRA = % de conteos con diferencia 0; 100 % si aún no hay conteos), faltantes (suma de discrepancias negativas) y sobrantes. El historial, en orden inverso, muestra fecha, producto, stock de sistema, físico, diferencia, motivo, si hubo ajuste, y acciones Editar / Revertir.

Nuevo conteo: se elige el SKU, se ve el stock del sistema, se ingresa lo contado (≥ 0) y un motivo (defecto «Ajuste de conteo»). Si hay diferencia, el motor inserta un movimiento de ajuste —venta si faltante, compra si sobrante— valorado al costo promedio de las capas vivas, o al costo base si no hay stock, y guarda el id en movimientoAjusteId. El motivo se antepone con «Faltante:» o «Sobrante:». Editar borra el movimiento previo, recalcula el stock limpio ($\text{sysSinAjuste} = \text{stock actual} - \text{discrepancia previa}$) y vuelve a insertar el ajuste. Revertir borra el conteo y su movimiento, como si nunca hubiera existido. El FIFO trata esos ajustes como cualquier compra o venta, de modo que el dashboard, el ABC y el plan de compras quedan consistentes.

6.7 Módulo 07 — Configuración

Umbral de días sin movimiento: Sana, Lenta y Muy lenta, con validación $0 \leq \text{Sana} \leq \text{Lenta} \leq \text{Muy lenta}$. Lo que supera el último corte es Dormido. Restablecer vuelve a 7 / 15 / 30. Datos del negocio: nombre (se muestra en el sidebar y en los informes) y apariencia (oscuro / claro). El método de costeo no es editable: FIFO es invariante. autoBackupMinutos existe en el JSON de configuración por compatibilidad de esquema; la copia automática real es diaria, en el proceso principal.

6.8 Módulo 08 — Copias de seguridad

Descargar copia completa exporta inventario_completo_AAAA-MM-DD.json (el estado entero, indentado). Cargar copia valida que existan productos, movimientos y config, y reemplaza la base. Ver copias automáticas lista hasta 20 archivos diarios del perfil Electron y permite Recuperar uno (reescribe Store y persiste). En el navegador no hay copias automáticas de disco: el aviso lo indica. PDF y Excel no se importan: son documentos de lectura.

6.9 Módulo 09 — Informes (PDF y Excel)

El operador elige el período del informe (Hoy ... Siempre), independiente del panel. Tres salidas:

- PDF — portada con banda petróleo, negocio, período y método; 1) resumen con cinco KPI y párrafo narrativo; 2) detalle por producto; 3) gráficos de impresión (barras y dona, paleta fija, canvas 880×420 , sin tocar los de pantalla); 4) clasificación ABC + Pareto; 5) qué comprar (solo SKU en alerta); 6) conteos e IRA (hasta 40 filas). Pie: «RotaStock Plus · FIFO (PEPS) · Página i de n». Archivo Informe_RotaStock_AAAA-MM-DD.pdf.

- Excel — 1. Capas FIFO (una fila por lote vivo, Valor capa = Stock × Costo unit. como fórmula, totales SUM, autofiltro A5:J); 2. Clasificación ABC (resumen de grupos + detalle); 3. Plan de compras; 4. Resumen (KPI, top 5 por ingresos, conteos). Archivo RotaStock_Informe_<período>_AAAA-MM-DD.xlsx.
- Imprimir — window.print() como respaldo si jsPDF falla.

Los gráficos de impresión usan tinta #1d211e, petróleo #1f5668, arcilla #a85a32 y semáforo de impresión (#38724f, #8a6810, #a9531a, #9c3630), independientes del tema de pantalla. La base ABC del Excel/PDF es la que esté seleccionada en el módulo 03.

7. Modelado matemático y algoritmos de negocio

7.1 Replay FIFO (PEPS)

Los movimientos se ordenan por timestamp (empate por id lexicográfico). Por producto se mantiene una cola de lotes { remaining, costo, fecha, movId }. Compra o ajuste_excedente empujan un lote. Venta (y faltante de conteo, persistido como venta con motivoAjuste) consume desde el frente:

$$COGS_{venta} = \sum take_k \cdot costo_k$$

$$Ingreso_{venta} = q \cdot precio$$

$take_k = \min(remaining_k, pendiente)$. El lote sale de la cola cuando $remaining \leq 10^{-9}$.

Si una venta supera los lotes (p. ej. se borró una compra), el residual se valora al último costo conocido (lastCost). Tras cada movimiento se actualizan stock, valor y un snapshot { fecha, qty, val } para interpolar inventario al inicio del período (valAt toma el último snapshot con fecha $\leq t$). Las capas vivas (capas[pid]) se exportan al Excel. Un movimiento cuyo productId ya no existe se ignora.

7.2 Ejemplo numérico de capas

Supóngase un SKU con tres documentos, en orden cronológico: compra de 10 uds. a 8; compra de 5 uds. a 10; venta de 12 uds. a precio 15. El replay construye dos lotes y consume el primero entero más 2 uds. del segundo:

Paso	Evento	Cola de lotes	COGS / ingreso
1	Compra 10 × 8	[10 @ 8]	—
2	Compra 5 × 10	[10 @ 8, 5 @ 10]	—
3	Venta 12 × 15	[3 @ 10]	COGS = 10·8 + 2·10 = 100; ingreso = 180

Tabla 4. Consumo PEPS. Stock final = 3, valor = 30, ganancia de la venta = 80.

Si más tarde un conteo físico reporta 2 uds., la discrepancia es -1 y se inserta una venta de ajuste al costo medio de las capas vivas ($30/3 = 10$). El IRA de ese conteo es 0; el stock pasa a 2 y el valor a 20. Revertir el conteo borra esa venta y restaura las 3 uds.

7.3 Rotación, cobertura y días de inventario

$$\text{Rotación} = \text{COGS_período} / ((I_0 + I_1) / 2)$$

$$\text{Cobertura (días)} = I_1 / (\text{COGS_período} / \Delta)$$

$$\text{Días de inventario} = (\Sigma \text{valor_actual}) / (\text{COGS_total} / \Delta)$$

$\Delta = \max(1, \text{días del rango})$. I_0 e I_1 son valor de inventario al inicio y al fin. Rotación = 0 si el promedio es 0; cobertura nula si no hay COGS; DIO nulo si totCogs = 0.

Engine.range(days) fija el intervalo: «hoy» recorta a las 00:00 locales; un entero N retrocede $N \times 86\,400\,000$ ms; 0 o falsy usa desde el movimiento más antiguo hasta ahora. Las entradas y salidas del período (entradasPer, salidasPer) se cuentan en unidades, no en dinero, y alimentan la hoja 1 del Excel.

7.4 Semaforización por inactividad

$$\text{días sin mov.} = \max(0, \lfloor (t_1 - \text{última_fecha}) / 86\,400\,000 \rfloor)$$

- Sana si $\text{días} \leq s$; Lenta si $s < \text{días} \leq \ell$; Muy lenta si $\ell < \text{días} \leq m$; Dormido si $\text{días} > m$; Sin datos si nunca hubo movimiento.

Fábrica: $s = 7$, $\ell = 15$, $m = 30$. El KPI de alertas y el banner cuentan solo Dormido. Un SKU puede tener rotación alta en el período y estar Dormido si lleva más de m días en silencio: velocidad histórica y silencio reciente son relojes distintos. El orden visual del semáforo (para ordenar la tabla) es sinDatos < sana < lenta < muyLenta < dormido.

7.5 Asignador heurístico ABC y estabilidad XYZ

Los productos se ordenan de mayor a menor según la base (valor en stock, o COGS con un ε de valor si $\text{COGS} = 0$). Umbrales clásicos 80 % y 95 %. Si un ítem cruza un umbral, no se corta a ciegas: se compara la distancia del acumulado incluyendo versus excluyendo respecto de la meta; el empate promueve a la clase superior. Eso evita que un SKU grande «robe» la clase A por un 0,1 % de exceso, y también evita dejarlo fuera cuando está más cerca de la meta incluyéndolo.

$$\text{decidir}(\text{antes}, \text{después}, \text{meta}) = 0 \text{ si } (\text{después} - \text{meta}) \leq (\text{meta} - \text{antes}), \text{ si no } 1$$

El índice de clase se resuelve así: si el acumulado posterior no supera 80, es A; si no supera 95, y el anterior ya había cruzado 80, es B; si el ítem salta un umbral, decide() elige. Un ítem que salta ambos umbrales de un golpe (antes < 80 y después > 95) prueba primero el 80 y, si no entra en A, el 95.

La estabilidad XYZ usa el coeficiente de variación de las ventas mensuales (cantidad, no dinero), agrupadas por YYYY-MM:

$$CV = \sigma_{\text{meses}} / \mu_{\text{meses}} \quad (0 \text{ si hay menos de 2 meses; } 9 \text{ si } \mu = 0)$$

- X Estable si $CV \leq 0,20$; Y Variable si $CV \leq 0,50$; Z Irregular en otro caso.

La cadencia de conteo sugerida sigue la clase ABC (A 30 / B 60 / C 120 días), no la XYZ: el dinero concentrado se cuenta más a menudo, aunque la demanda sea irregular.

7.6 Plan de compras (punto de pedido)

Demanda diaria media $\bar{d}$ = (promedio de unidades/mes) / 30. Desvío sd_D sobre esas tasas diarias (o $0,3 \cdot \bar{d}$ si hay un solo mes). Pronóstico mensual F con suavizado exponencial $\alpha = 0,3$ sobre la serie mensual ordenada. Lead time LT (días, defecto 7), desvío de lead time s_LT (defecto 1) y factor Z de servicio (1,65 / 1,28 / 1,04):

$$F_t = \alpha \cdot D_t + (1 - \alpha) \cdot F_{t-1} \quad \alpha = 0,3$$

$$SS = Z \cdot \sqrt{LT \cdot sd_D^2 + \bar{d}^2 \cdot s_LT^2}$$

$$PRO = \bar{d} \cdot LT + SS$$

$$\text{sugerido} = \max(0, \lceil PRO - \text{stock_actual} \rceil)$$

Alerta «Comprar» si $\text{stock_actual} \leq PRO$. El modelo asume demanda y lead time independientes. $F = 0$ si no hay ventas.

El radicando se recorta a ≥ 0 para no producir NaN. El suavizado arranca en el primer mes observado. El plan se calcula con compute(0): usa el stock actual (todas las capas vivas), no el stock interpolado al inicio de un período corto.

7.7 Ajustes de conteo e IRA

$$\text{discrepancia} = \text{físico} - \text{sistema}$$

$$IRA = (\text{conteos con discrepancia} = 0) / N \times 100$$

Si discrepancia < 0 se registra una venta de $|d|$ unidades al costo medio de capas (faltante). Si > 0, una compra al mismo costo (sobrante). El motivo se guarda en

motivoAjuste. El flag iraCumplido (1 si $d = 0$) queda en el registro del conteo; el KPI usa la discrepancia, no ese flag. Editar un conteo borra el movimiento previo, recalcula el stock limpio y vuelve a insertar el ajuste. Revertir elimina ambos. Así el IRA, el FIFO y el plan de compras hablan el mismo idioma.

7.8 Orden de recálculo

renderAll() es el único punto que refresca las nueve vistas. Secuencia: COMP = Engine.compute(PERIOD); se pinta el pie del período y el sidebar; luego renderDash, renderProds, renderABC, renderMovs, renderReab, renderAud, renderCfg. Cada mutación (alta, edición, baja, conteo, config, importación) hace Store.save() y llama renderAll(). Actualizar (cabecera) solo re-renderiza, no relée el disco. El período del panel, al cambiar, reinicia la página del detalle a 1.

8. Flujo de operación

9. Instalar RotaStockPlus Setup 2.0.0.exe (o abrir el portable). Los datos se crean solos en el perfil de Windows.
10. En Configuración, poner el nombre del negocio y el tema. Ajustar umbrales si el almacén no es de consumo diario.
11. Registrar productos. Si hay existencias, declarar unidades iniciales y el costo total del lote: eso abre la primera capa FIFO.
12. Registrar compras y ventas. El sistema rechaza una venta sin stock. El precio de venta no altera el COGS.
13. Revisar el Panel: DIO, margen, productos dormidos y el detalle ordenable.
14. Consultar Clasificación ABC (por dinero o por COGS) para saber qué contar primero.
15. Antes de pedir al proveedor, abrir Qué comprar: lead time y Z del producto ya alimentan el punto de pedido.
16. Contar el almacén (prioridad A cada 30 días). Si hay diferencia, el ajuste queda en la bitácora; no se parchea el stock a mano.
17. Generar el PDF o el Excel del período que se quiere reportar—independiente del lente del Panel.
18. Antes de cambiar de PC, Descargar copia completa. En la otra máquina, Cargar copia.

9. Directrices de operación y mantenimiento

- Registrar primero el producto y después sus movimientos. Una venta sin stock se rechaza.
- El «costo» de una venta es el precio. El COGS lo calcula FIFO, no el operador.
- Stock inicial: el costo total es del lote completo; el sistema divide.
- Lead time y margen Z se editan en el producto: alimentan «Qué comprar», no el FIFO.
- Ajustar umbrales al ritmo del almacén: repuestos lentos necesitan s, ℓ , m más holgados.
- Contar los A cada 30 días: es donde está el dinero.
- Un conteo con diferencia genera un movimiento visible en la bitácora; no lo borre a mano, revértalo desde Conteos.
- El período del panel y el del informe son independientes: elija cada uno según la pregunta.
- Antes de cambiar de PC, descargue la copia completa JSON. El PDF/Excel no restauran el sistema.
- No edite inventario.json a mano. Use los módulos y las copias automáticas diarias.
- Librerías vendorizadas: no hace falta red para graficar, ni para PDF, ni para Excel.
- Mantenimiento: umbrales por defecto, $\alpha = 0,3$, Z, s_LT y retención de 30 días viven en app.js y main.js. No hay archivo .env.
- No confíe en claseABC persistida: siempre manda Engine.abc().
- En navegador, trate localStorage como borrador; la copia JSON es la que viaja de máquina a máquina.

10. Conclusiones

- Un JSON, un replay, nueve módulos: RotaStock Plus cubre el ciclo del almacén sin servidor.
- FIFO como invariante: cada venta consume el lote más antiguo y deja un COGS auditable.
- ABC heurístico por proximidad evita cortes rígidos 80/95; XYZ describe la regularidad de la demanda.
- El plan de compras convierte lead time y servicio Z en un punto de pedido operable.
- Los conteos no parchean el stock: insertan o revierten movimientos, y el IRA mide la disciplina.
- PDF y Excel hablan el mismo idioma que la pantalla, con período propio y fórmulas vivas en las capas FIFO.

- Portabilidad real: instalador o .exe, perfil de usuario, copia diaria y un archivo para migrar de PC.
- El renderer está aislado del disco. Cuatro canales IPC y un basename bastan para no salir de AppData.

Anexo A. Esquema JSON de estado único

Campo	Tipo	Rol
meta.sistema / versionEsquema	texto	«RotaStock Plus» / «2.0.0»
meta.createdAt	ISO-8601	Alta del archivo
meta.totalProductos / totalMovimientos	enteros	Se reescriben en cada save()
config.metodoCosteo	«FIFO»	Constante; no editable en UI
config.umbrales.*	enteros (días)	Cortes del semáforo
config.preferenciasUI.tema	claro oscuro	Apariencia
config.preferenciasUI.nombreEmpresa	texto	Sidebar e informes
productos[].id / nombre	string	Clave y etiqueta
productos[].leadTime / desviacionLeadTime	número	Plan de compras (def. 7 y 1)
productos[].nivelServicioZ	1.65 1.28 1.04	Alto / Normal / Bajo
productos[].costoUnitario	número	Costo base (faltante de capas)
productos[].creadoEn	ISO-8601	Alta del SKU
movimientos[].tipo	compra venta	Dirección; el motor también acepta ajuste_excedente
movimientos[].cantidad / costoUnitario / timestamp	entero / número / ISO	Documento
movimientos[].motivoAjuste	texto null	Origen de un conteo (Faltante:/Sobrante:)
movimientos[]._cogs / _ingreso	número derivado	Se reescriben en compute()
conteos[].stockSistema / stockFisico	enteros	Antes y después del conteo
conteos[].discrepancia	entero	físico – sistema
conteos[].iraCumplido	0 1	1 si discrepancia = 0
conteos[].movimientoAjusteId	id null	Vínculo al movimiento de ajuste

Tabla 5. Esquema de inventario.json. Los lotes no se persisten: viven solo en Engine.compute(). En el archivo el arreglo de conteos se llama conteosCiclicos.

Anexo B. Correspondencia interfaz–motor

Elemento de pantalla	Función / identificador	Expresión
Días de inventario	kpiDias / COMP.dio	Σ valor / (COGS / Δ)
COGS / Ingresos / Ganancia	kpiCogs, kpiIngresos, kpiGanancia	Flujos del período
Alertas	kpiAlertas / COMP.dorm	SKU con estado dormido
Grupo A/B/C	Engine.abc	Proximidad a 80 % y 95 %
Estabilidad	Engine.cv	CV mensual $\rightarrow$ X/Y/Z
Pedir al llegar a	PRO	$\bar{d}$ ·LT + SS
Cantidad a pedir	sug	[PRO – stock]
Exactitud de conteos	IRA	% de diferencias nulas
PDF secc. 1–6	buildPdf + chartPNG	jsPDF + gráficos de impresión
Excel hoja 1	buildWorkbook	Valor capa = Stock×Costo (fórmula)
Copia completa	exportJSON / fileImport	JSON indentado, validado al cargar
Tema	applyTheme	claro $\leftrightarrow$ dark, rotastock_theme

Tabla 6. Trazabilidad entre la interfaz y el motor.

Anexo C. Validaciones de formularios

Formulario	Regla	Si falla
Nuevo producto	Nombre no vacío	Toast «Ponle un nombre»
Nuevo producto	Si unidades > 0, costo total > 0	Toast «Falta el costo total»
Movimiento	Producto elegido (id), cant $\geq$ 1, precio $\geq$ 0, fecha válida	Toast «Revisa los datos»
Venta	cant $\leq$ stock FIFO (más reintegro si se edita)	Toast con disponible
Conteo	Físico $\geq$ 0 y producto existente	Toast «Ingresa lo contado»
Umbrales	$0 \leq$ sana $\leq$ lenta $\leq$ muyLenta	Toast «de menor a mayor»
Cargar copia	Existen productos, movimientos y config	Toast «no es una copia válida»
readBackup	path.basename(nombre)	No se sale de backups/

Tabla 7. Validaciones. Ninguna mutación se persiste si la regla no se cumple.

Anexo D. Estructura de los informes PDF y Excel

PDF (A4, mm)	Contenido
Banda 0–30 mm	Título, negocio, período, método FIFO, fecha de emisión
§1 Resumen	Cinco cajas KPI + párrafo narrativo de ventas, costo, ganancia, DIO y alertas
§2 Detalle	Tabla: producto, stock, inv. prom., rotación, cobertura, sin mov., estado
§3 Gráficos	Barras top-8 e dona de salud, paleta de impresión
§4 Clasificación	Párrafo A/B/C, tabla y curva Pareto
§5 Qué comprar	Solo SKU en alerta; si no hay, un párrafo de confirmación
§6 Conteos	IRA, faltantes, sobrantes y hasta 40 filas
Pie	RotaStock Plus · FIFO (PEPS) · Página i de n

Tabla 8. Informe PDF (buildPdf).

Hoja Excel	Contenido y fórmulas
1. Capas FIFO	Una fila por lote vivo. Valor capa = Stock × Costo (fórmula). Totales SUM de entradas, salidas, stock y valor. Autofiltro.
2. Clasificación ABC	Resumen A/B/C con % del dinero; detalle con % total, % acumulado, grupo, estabilidad y cadencia.
3. Plan de compras	Stock, $\bar{d}$, pronóstico, SS, PRO, sugerido, Comprar/Bien.
4. Resumen	KPI del período, top 5 por ingresos, conteos, IRA, faltantes y sobrantes.

Tabla 9. Libro Excel (buildWorkbook). Formatos: money #,##0.00; int; qty1; pct1; fecha yyyy-mm-dd.

Anexo E. Canales IPC y empaquetado

electron-builder genera dos artefactos x64: el instalador NSIS (directorio cambiante, icono propio, accesos directos «RotaStockPlus» en escritorio e inicio) y el portable RotaStockPlus.exe. Los files del paquete son main.js, preload.js, package.json y frontend/**; datos/ y backups/ no viajan. La semilla, si existe al primer arranque junto al ejecutable desempaquetado, se copia; en el asar empaquetado se usa defaultDB si no hay perfil previo.

app.js exporta { Engine, Store, buildWorkbook, buildPdf } cuando typeof module !== 'undefined', de modo que el motor puede probarse fuera de la ventana. El frontend no consulta red en operación: las cuatro librerías vendor cubren gráficos, PDF y Excel.

— Fin de la Memoria Descriptiva —

RotaStock Plus · Pedro Giovanni Ricra Figueroa · 2026